

Proyecto 01

 2026-01

Qué hay que hacer?

Construir una **aplicación React de múltiples páginas (SPA)** que integre **todos los conceptos** vistos hasta ahora.

La **imagen de referencia** que se les suministra es una **guía de qué vistas** deben implementar — **no** de cómo se deben ver. El diseño visual (colores, tipografía, layout específico) es **libre**.

Ustedes deciden cómo organizar el código, qué componentes crear, y cómo nombrarlos. Lo que se evalúa es **qué funciona**, no cómo lo estructuraron.

La API

Tres opciones:

1. **Escoger** una de la lista sugerida (públicas, gratuitas, sin API key)
2. **Usar otra API** pública que ustedes encuentren
3. **Construir su propia API** (Express, FastAPI, json-server, etc.)

No se aceptan datos hardcodedos — los datos deben venir de una API real.

API	URL base	Datos disponibles
Rick and Morty	https://rickandmortyapi.com/api/character	Personajes, imágenes, estado, especie
REST Countries	https://restcountries.com/v3.1/all	Países, banderas, capitales, población, región
DummyJSON Products	https://dummyjson.com/products	Productos, imágenes, precio, categoría, rating
TVMaze	https://api.tvmaze.com/shows	Series TV, imágenes, géneros, rating, sinopsis
Jikan (MyAnimeList)	https://api.jikan.moe/v4/anime	Anime, imágenes, score, géneros, sinopsis

API	URL base	Datos disponibles
TheMealDB	<code>https://www.themealdb.com/api/json/v1/1/search.php?s=</code>	Recetas, imágenes, categorías, instrucciones
Art Institute of Chicago	<code>https://api.artic.edu/api/v1/artworks</code>	Obras de arte, artista, fecha, imágenes
PokeAPI	<code>https://pokeapi.co/api/v2/pokemon?limit=50</code>	Pokémon, sprites, tipos, stats
Digimon	<code>https://digimon-api.vercel.app/api/digimon</code>	Digimon, imágenes, nivel
SpaceX	<code>https://api.spacexdata.com/v4/launches</code>	Lanzamientos, nombres, fechas, parches

Requisitos técnicos

✓ Proyecto creado con **Vite + React**

✓ **React Router** con sus rutas

- Página de inicio (landing/hero)
- Página de exploración (grid de tarjetas con fetch de lista)
- Página de detalle (fetch individual por ID, ruta dinámica)
- Página de favoritos (items guardados en estado)
- Página de contacto (formulario controlado)
- Página 404

✓ **Dos fetches:** lista y detalle por ID

Requisitos técnicos

- ✓ Estados visibles en la UI:
 -  **Loading** — mientras la petición está en curso
 -  **Error** — si la petición falla
 -  **Empty** — cuando no hay resultados (ej. búsqueda sin matches, favoritos vacíos)
- ✓ **Búsqueda y filtrado** funcional en tiempo real
- ✓ **Favoritos** en estado (agregar/quitar, persisten entre rutas)
- ✓ **Formulario controlado** con validación (nombre, email, mensaje) y botón disabled

Requisitos técnicos

- ✓ Toast de éxito, error e info con auto-dismiss
- ✓ Modal de confirmación con `<dialog>` nativo + `useRef` + `showModal()` — para confirmar acciones (quitar de favoritos, enviar formulario, etc.)
- ✓ Modal con dark pattern intencional + comentario explicativo en el código
- ✓ SVG inline (logo o icono, directamente en JSX)
- ✓ Tailwind CSS para todo el styling — responsive con prefijos
- ✓ Accesibilidad: `alt` , `role` , `aria-label` , `htmlFor` , HTML semántico
- ✓ Mínimo 8 componentes en archivos `.jsx` separados

Contexto

Revisen los temas ya vistos en clase:

- **Routing:** React Router, `useParams` , `NavLink`
- **Fetching:** `useEffect` , `fetch` , `async/await` , estados de loading/error/empty
- **Formularios:** controlados, validación, `useState`
- **Dialogs:** `useRef` , `<dialog>` , `showModal()` , dark patterns
- **Estilos:** Tailwind CSS, responsive, grid, utilidades
- **Accesibilidad:** SVG inline, `alt` , `role` , `aria-label` , HTML semántico

Para los **favoritos**: piensen en dónde debe vivir ese estado para que persista entre rutas.

Para el **toast**: pueden usar una librería como `react-hot-toast` , `sonner` , o `react-toastify` (`npm install`). También pueden implementarlo desde cero con `useState` + `setTimeout` .

Entrega

 **Formato:** código en **GitHub** + app desplegada en **Vercel**

 **Fecha límite:** según lo indicado en el aula virtual

 **Cómo entregar:** subir al aula virtual dos enlaces:

1.  URL del repositorio de **GitHub** (público)
2.  URL de la app desplegada en **Vercel**

 **No se aceptan archivos** `.zip`

 **Importante:**

- Este es un proyecto **individual**
- Todas las **vistas de la referencia** deben estar implementadas (el diseño visual es libre)

